

CSE 112 – Lab #4 Corporate Sales – Constructors, Classes, Input Validation, and Test Cases

Modified Programming Challenge #10 (page 938) with [Test Cases \(10%\)](#) and file reading.

Implement the DivSales class that holds sales data for a division, with the array for the sales data by quarter, and a private static variable that holds the total sales for all sales divisions.

Do not ask the user to enter the sales data. The program will create the **array of six objects**, and read the sales data below into the objects from a **file called 'SalesFile.txt'**. The program will then prompt the user for the division number (1 – 6) and output the sales for that division, and the total sales for that division as shown below. Use functions as you see fit. **No input validation is required.**

Add a loop in main (do not use while(true), while(1), break or continue) so that multiple numbers can be input and "0" to quit. When the user enters '0' to quit, output the total sales for the corporation (the private static variable value).

Complete a [Test Case](#) (10%) document with tests and the outcomes when you ran the tests (see below).

The program will use the class definition provided below in a separate *.h file, and use the class member functions to obtain the proper output (including the private static variable). Do not change the class.

```
// Declaration of the DivSales class for the *.h file
```

```
class DivSales
```

```
{
```

```
    private:
```

```
        static double corpSales;
```

```
        double divSales;
```

```
    public:
```

```
        double qtrs[4];
```

```
        void setDivSales(double s)
```

```
        { divSales = divSales + s; }
```

```
        void setCorpSales(double s)
```

```
        { corpSales = corpSales + s; }
```

```
        double getDivSales() const
```

```
        { return divSales; }
```

```
        double getCorpSales() const
```

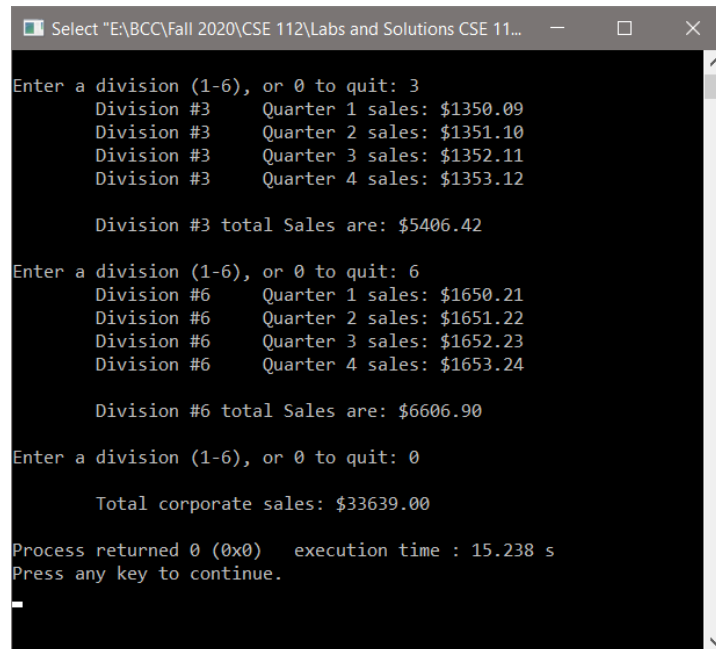
```
        { return corpSales; }
```

```
};
```

```
// Static member variable definition
```

```
double DivSales::corpSales;
```

Output Format – tabs and spacing required:



```
Select "E:\BCC\Fall 2020\CSE 112\Labs and Solutions CSE 11..."
Enter a division (1-6), or 0 to quit: 3
Division #3    Quarter 1 sales: $1350.09
Division #3    Quarter 2 sales: $1351.10
Division #3    Quarter 3 sales: $1352.11
Division #3    Quarter 4 sales: $1353.12

Division #3 total Sales are: $5406.42

Enter a division (1-6), or 0 to quit: 6
Division #6    Quarter 1 sales: $1650.21
Division #6    Quarter 2 sales: $1651.22
Division #6    Quarter 3 sales: $1652.23
Division #6    Quarter 4 sales: $1653.24

Division #6 total Sales are: $6606.90

Enter a division (1-6), or 0 to quit: 0

Total corporate sales: $33639.00

Process returned 0 (0x0) execution time : 15.238 s
Press any key to continue.
```

Date for "SalesFile.txt"

1150.01
1151.02
1152.03
1153.04
1250.05
1251.06
1252.07
1253.08
1350.09
1351.10
1352.11
1353.12
1450.13
1451.14
1452.15
1453.16
1550.17
1551.18
1552.19
1553.20
1650.21
1651.22
1652.23
1653.24

Test Case Document: A test case/results document is required for this project (10% of grade).

What is a Test case?

A test case describes input, action, and the result of running the test, in order to determine if a feature or requirement of an application is working correctly.

A test case is a set of instructions on “HOW” to validate a particular test objective/target, which when followed will determine if the expected behavior of the system is satisfied or not.

List your test cases, run them, and note the results

Template:

Engineer Name: *Your Name*

Project/program description: *A brief description of program operation (what does it do?)*

Test Case #1: *describe the test*

Expected Test Results:

Actual Test Results:

Test Case #2: *describe the test*

Expected Test Results:

Actual Test Results:

Test Case #3: *describe the test*

Expected Test Results:

Actual Test Results:

-

-

Test Case #n: *describe the test*

Expected Test Results:

Actual Test Results: